

Отчёт по практической работе 1: Jenkins

Студент: Козлова Анастасия Витальевна, red28

Задание 1. Создание первого Job

Цель

Ознакомиться с интерфейсом Jenkins, научиться создавать простейший проект (Freestyle project), настраивать шаги сборки с использованием Shell-скриптов, запускать задачи и анализировать результат через Console Output.

Выполнение

1. На главной странице Jenkins нажата кнопка «+ New Item».
2. Создан новый Item с именем myJenkinsJob_28, тип — Freestyle project.
3. В поле Description добавлено: «Мой первый jenkins job в рамках курса КИС».
4. В разделе Build Steps добавлен шаг Execute shell с командой:
5. `bash`
6. `echo "Hello World!"`
7. Job сохранён и запущен.

Результат

Скриншот №1: Console Output задания myJenkinsJob_28

The image displays two screenshots of the Jenkins web interface, showing the execution and completion of a job named 'myJenkinsJob_28'.

Top Screenshot: Console View

- The browser address bar shows the Jenkins URL: `89.169.185.172:8080 Console - ...`.
- The Jenkins header shows the job name: **myJenkinsJob_28**.
- The left sidebar contains navigation options: Статус, Изменения, Вывод консоли (selected), Редактировать информацию сборки, Delete build '#1', and Timings.
- The main area is titled **Console** and displays the build output:

```
Started by user red28
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/myJenkinsJob_28
[myJenkinsJob_28] $ /bin/sh -xe /tmp/jenkins6915090022463961715.sh
+ echo Hello World!
Hello World!
Finished: SUCCESS
```
- Buttons for **Download**, **Copy**, and **View as plain text** are visible.
- The bottom right corner indicates **Jenkins 2.555.1**.

Bottom Screenshot: Job Overview View

- The browser address bar shows the Jenkins URL: `89.169.185.172:8080 myJenkinsJob_28 - Jenkins`.
- The Jenkins header shows the job name: **myJenkinsJob_28**.
- The left sidebar contains navigation options: Статус, Изменения, Сборочная директория, Собрать сейчас, Настройки, Удалить Проект, and Переименовать.
- The main area shows the job status as **myJenkinsJob_28** with a green checkmark icon. Below it, the description reads: **Мой первый jenkins job в рамках курса КИС**.
- The **Постоянные ссылки** (Permanent links) section lists recent builds:
 - Последняя сборка (#1), 55 секунд назад
 - Последняя стабильная сборка (#1), 55 секунд назад
 - Последняя успешная сборка (#1), 55 секунд назад
 - Last completed build (#1), 55 секунд назад
- The **Builds** section shows a list of builds with a search filter: **Фильтровать сборки...**. The first build is marked as successful (green checkmark) and labeled **#1 07:03**.
- The bottom right corner indicates **REST API** and **Jenkins 2.555.1**.

Job выполнен успешно, в консоли выведено «Hello World!».

Задание 2. Создание веб-страницы

Цель

Научиться создавать многошаговые пайплайны, включающие генерацию артефактов (HTML-страницы) и их тестирование, а также настроить ротацию логов (удаление старых сборок через 5 дней)

Выполнение

1. Создан новый Job с именем WebPage_28, тип — Freestyle project.
2. В разделе «Discard old builds» включена ротация логов: Days to keep builds = 5.
3. Добавлен первый шаг Execute shell для создания HTML-страницы:
4. `bash`

```
echo "---Prom Env Build Start---"
cat <<EOF > index.html
<html>
<body bgcolor="yellow">
<center>
<h1><font face="Arial" color="black">Hello World</font></h1>
<p><font face="Arial" color="green">My first Test Env Build</font>
</center>
</body>
</html>
EOF
```

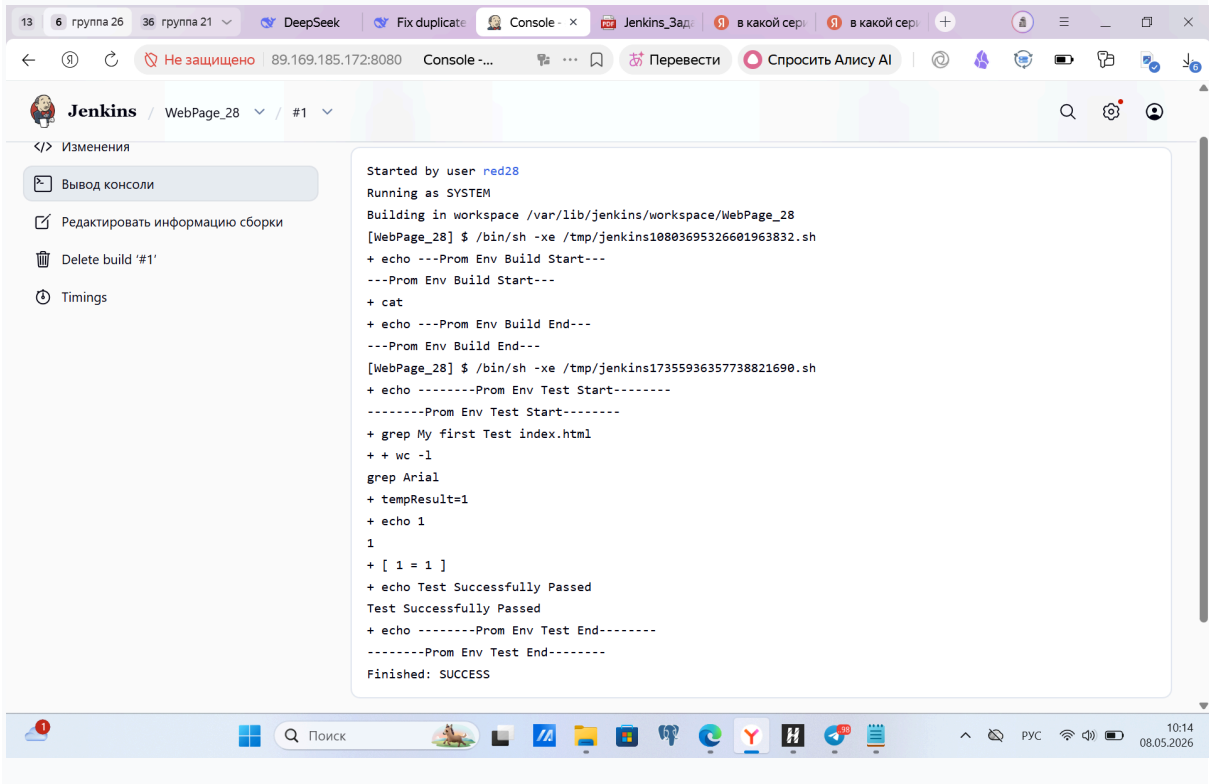
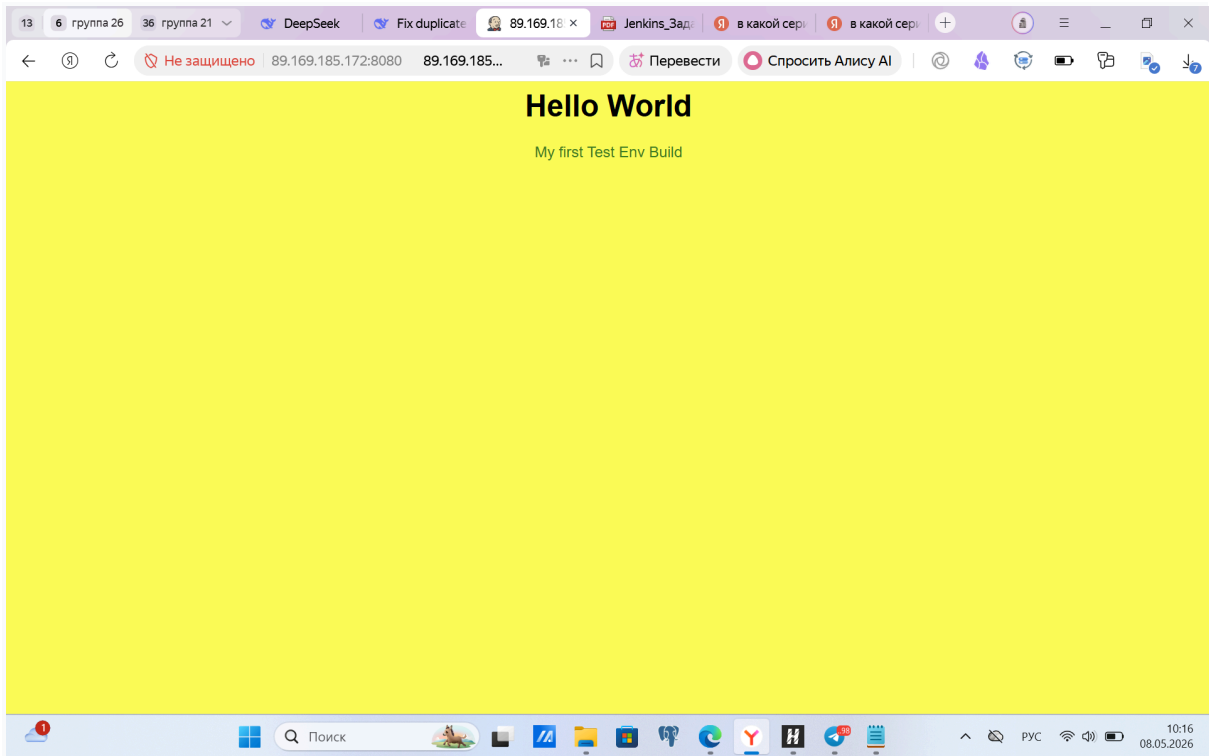
5. `echo "---Prom Env Build End---"`
6. Добавлен второй шаг Execute shell для тестирования:
7. `bash`

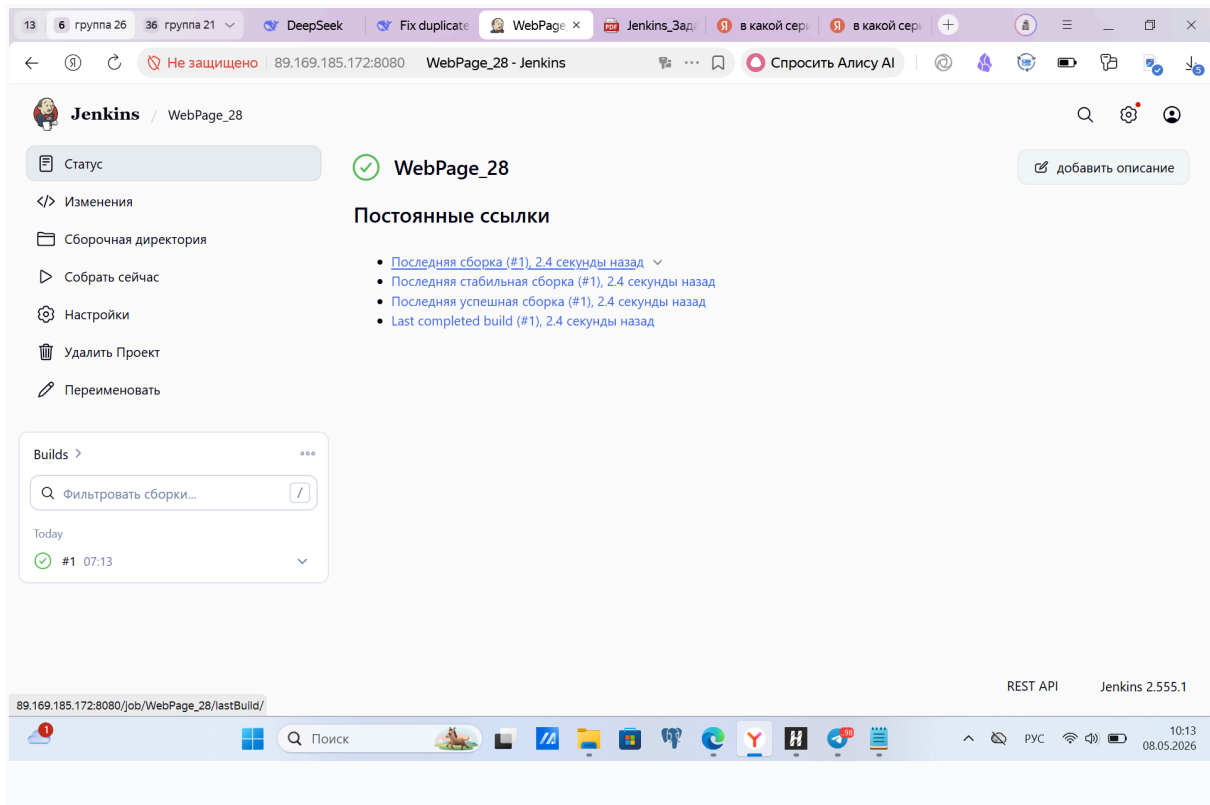
```
echo "-----Prom Env Test Start-----"
tempResult=`grep "My first Test" index.html | grep "Arial" | wc -l`
echo $tempResult
if [ "$tempResult" = "1" ]; then
    echo "Test Successfully Passed"
else
    echo "Test Failed"
    exit 1
fi
```

8. `echo "-----Prom Env Test End-----"`

Результат — успешный тест

Скриншот №2: Console Output с успешным тестом





Результат — падение теста после изменения

Для проверки работоспособности теста в первом Execute shell слово `Arial` заменено на `Test`:

```
html
```

```
<p><font face="Test" color="green">My first Test Env Build</font>
```

После повторного запуска тест упал:

Скриншот №3: Console Output с упавшим тестом

The top screenshot shows the Jenkins console output for a build named 'WebPage_28'. The output indicates that the build failed due to a shell script error. The console text is as follows:

```
Started by user red28
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/WebPage_28
[WebPage_28] $ /bin/sh -xe /tmp/jenkins13708566444403194675.sh
+ echo ---Prom Env Build Start---
---Prom Env Build Start---
+ cat
+ echo ---Prom Env Build End---
---Prom Env Build End---
[WebPage_28] $ /bin/sh -xe /tmp/jenkins17813424735883441462.sh
+ echo -----Prom Env Test Start-----
-----Prom Env Test Start-----
+ grep My first Test index.html
+ grep Arrial
+ wc -l
+ tempResult=0
+ echo 0
0
+ [ 0 = 1 ]
+ echo Test Failed
Test Failed
+ exit 1
Build step 'Execute shell' marked build as failure
Finished: FAILURE
```

The bottom screenshot shows the Jenkins build status page for 'WebPage_28'. The page displays a list of builds under the 'Постоянные ссылки' (Permanent links) section. The builds are as follows:

- Последняя сборка (#2), 2.6 секунды назад
- Последняя стабильная сборка (#1), 6 минут 16 секунд назад
- Последняя успешная сборка (#1), 6 минут 16 секунд назад
- Последняя провалившаяся сборка (#2), 2.6 секунды назад
- Last unsuccessful build (#2), 2.6 секунды назад
- Last completed build (#2), 2.6 секунды назад

The 'Builds' section shows a list of builds with their status and timestamps:

Build	Status	Timestamp
#2	Failed	07:20
#1	Successful	07:13

Тест корректно обнаружил изменение содержимого.

Задание 3. Развертывание Python-сервера

Цель

Научиться использовать параметризацию сборок для динамической настройки параметров приложения (порт на основе STUDENT_ID), освоить контейнеризацию: от генерации Dockerfile до запуска Python-сервера в Docker.

Выполнение

1. Создан новый Pipeline с именем web-server-job_28.
2. Включена параметризация: добавлен строковый параметр STUDENT_ID.
3. В поле Pipeline script вставлен код (предоставленный в задании), который:
 - Проверяет наличие STUDENT_ID
 - Вычисляет порт по формуле $70\$\{STUDENT_ID\}$
 - Создает файлы `app.py` и `Dockerfile`
 - Останавливает и удаляет старый контейнер (если есть)
 - Собирает Docker-образ и запускает контейнер
4. Запуск произведён с параметром `STUDENT_ID=28`.
5. Порт вычислен: `7028`.

Результат

Скриншот №4: Console Output Pipeline

The screenshot shows the Jenkins web application running in a browser. The address bar displays "https://89.169.185.172:8080". The main header includes the Jenkins logo and the breadcrumb "web-server-job_28". On the left sidebar, navigation links include Status, Changes, Собрать с параметрами (highlighted), Настройки, Удалить Pipeline, Stages, Переименовать, and Pipeline Syntax. The main content area is titled "Pipeline web-server-job_28" and contains a note: "Для этой сборки необходимы следующие параметры:". Below this, there is a parameter definition for "STUDENT_ID" with a value field containing "28". At the bottom of the configuration section are two buttons: "Собрать" (Build) and "Cancel". A "Builds" section at the bottom left shows "Нет сборок" (No builds). The footer indicates "Jenkins 2.555.1".

13 8 rpyhna 26 36 rpyhna 21 DeepSe Fix dup web x Jenkins в какой в какой Gitlab 89.169

← ① ↻ Не защищено 89.169.185.172:8080 web-server-job_28 #1 - Jenkins Спросить Алису AI

Jenkins / web-server-job_28 / #1

Status

</> Changes

Console Output

Edit Build Information

Delete build '#1'

Параметры

Timings

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

✓ #1 (8 мая 2026 г., 07:33:03)

добавить описание

Хранить эту сборку вечно

Создана пользователем red28

Запущена 6 минут 24 секунд назад

Заняло: 5 секунды

This run spent:

- 6 ms waiting;
- 5 секунды build duration;
- 5 секунды total from scheduled to completion.

</> Нет изменений.

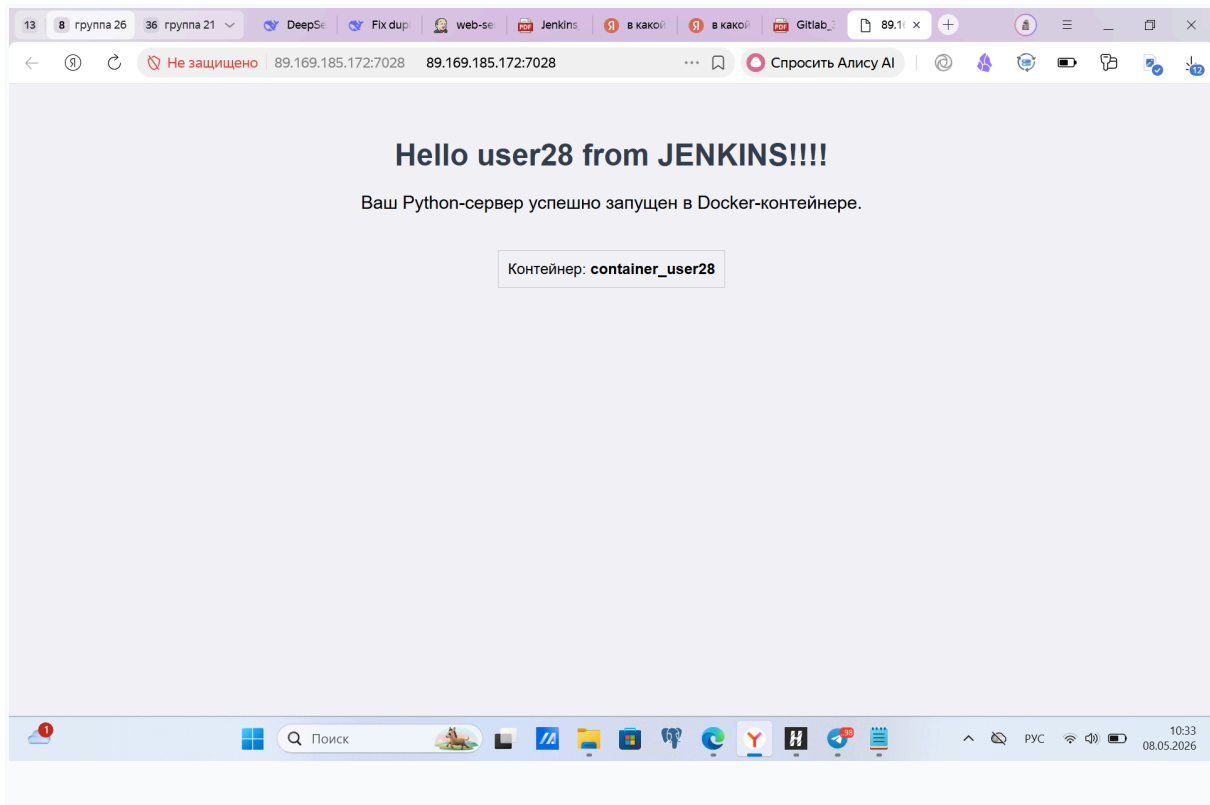
REST API Jenkins 2.555.1

13 8 rpyhna 26 36 rpyhna 21 DeepSe Fix dup Console x Jenkins в какой в какой Gitlab 89.169

← ① ↻ Не защищено 89.169.185.172:8080 Console... Перевести Спросить Алису AI

Jenkins / web-server-job_28 / #1

```
+ docker run -d --name container_user28 -e USER_NAME=user28 -p 7028:8080 img_user28
ec57426722968b7f362b52d3b071754cf1bdb0cab57576d6be52f723b010e4c1
[Pipeline] echo
Контейнер запущен на порту 7028
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
---
[Pipeline] echo
ЗАДАНИЕ ВЫПОЛНЕНО УСПЕШНО!
[Pipeline] echo
Ваша ссылка: http://89.169.185.172:7028
[Pipeline] echo
---
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Скриншот №5: Страница в браузере по адресу `http://89.169.185.172:7028`

Python-сервер успешно развёрнут в Docker-контейнере и доступен по внешнему адресу.

Ответы на контрольные вопросы

1. С какими трудностями вы столкнулись при анализе Console Output и как логи помогают в поиске ошибок?

В целом глобальных трудностей нет, очень удобны комментарии в коде, поняла как важно их писать

Мелкие трудности:

- Большой объём вывода (особенно при сборке Docker), в котором легко пропустить важное
- Не всегда понятно, где заканчиваются успешные шаги и начинается ошибка и не всегда понятно что происходит конкретно
- Ошибки в скриптах не всегда очевидны из вывода, если команда «успешно» выполнялась, но результат не тот

Логи при этом конечно помогают, они показывают выполнение команд по шагам, при поиске ошибок если разбираться во всех командах и их обозначениях можно проследить путь ошибки и выявить ее

2. Какую роль в Задании №3 выполнял Docker, и почему сервер стал доступен по внешнему адресу только после этапа Run Container?

Docker создаёт изолированный контейнер, внутри которого работает Python-сервер. Благодаря этому:

- Приложение не зависит от библиотек и версий Python на хосте
- Обеспечивается воспроизводимость среды

Почему сервер стал доступен только после Run Container

Сервер стал доступен только после выполнения команды Run Container потому, что на этапе Create Files создаются лишь исходные файлы, например app.py и Dockerfile, без какого-либо запуска самого приложения, тогда как на этапе Build & Run сначала выполняется docker build, который формирует образ, содержащий Python-код, а затем уже docker run, который непосредственно запускает контейнер, и только после этого запуска приложение начинает слушать порт и обрабатывать HTTP-запросы, причём использование ключа -p 7028:8080 обеспечивает проброс порта, связывая внутренний порт контейнера 8080 с внешним портом хоста 7028, что и делает сервер доступным извне

3. Зачем мы использовали STUDENT_ID в параметрах сборки?

Что произошло бы, если бы все студенты запустили проект без этого параметра?

Это необходимо для уникальности - каждый студент запускает контейнер на своём порту и для изоляции ресурсов - имена контейнеров и образов содержат ID студента, что исключает конфликты

Что произошло бы без параметра:

- Все студенты запускали бы контейнер на одном и том же порту
- Контейнеры пытались бы занять один порт → ошибка бы возникала что адрес уже используется

- Имена контейнеров совпадали бы (`container_user`) и Docker не смог бы запустить второй контейнер с тем же именем

Вывод: параметр обеспечивает многопользовательскую изоляцию — ключевое требование для учебной среды, где много студентов работают одновременно.

Общий вывод

- Jenkins настроен и работает.
- Созданы три типа заданий: Freestyle с shell-командами, Freestyle с генерацией и тестированием HTML, параметризованный Pipeline с Docker.
- Получены практические навыки работы с Jenkins, анализом логов, контейнеризацией и параметризацией сборок.